

# **SentinelShield: Advanced Intrusion Detection & Web Protection System**

**TECHNICAL DOCUMENTATION REPORT**

**Submitted by:**

**UJWALA CHIPPADA**

## Table of Contents

|                                                              |    |
|--------------------------------------------------------------|----|
| 1. Project Overview / Description .....                      | 04 |
| 2. Project Scope and Practical Objectives .....              | 05 |
| 3. Practical Concepts and Background .....                   | 06 |
| 3.1 HTTP Request Inspection .....                            | 07 |
| 3.2 Attack Signature Identification .....                    | 08 |
| 3.3 Behavior Monitoring and Rate Limiting .....              | 09 |
| 3.4 Log Analysis .....                                       | 10 |
| 3.5 Dashboard Interpretation .....                           | 12 |
| 4. Practical Workflow .....                                  | 14 |
| 4.1 Step 1: Understanding the System Architecture .....      | 15 |
| 4.2 Step 2: Reviewing Rule Definitions .....                 | 15 |
| 4.3 Step 3: Simulating HTTP Requests .....                   | 15 |
| 4.4 Step 4: Observing Detection Behavior .....               | 15 |
| 4.5 Step 5: Log File Examination .....                       | 15 |
| 4.6 Step 6: Reporting and Analysis .....                     | 15 |
| 5. Detailed System Working (Non-Technical Explanation) ..... | 16 |
| 5.1 Request Processing .....                                 | 16 |
| 5.2 Rule-Based Detection .....                               | 16 |
| 5.3 Traffic Monitoring & Sliding Windows .....               | 17 |
| 5.4 Alert Decisions Sequence .....                           | 17 |

|                                          |    |
|------------------------------------------|----|
| 5.5 Logging & Dashboard Visibility ..... | 17 |
| 6. Practical Output .....                | 18 |

## 1. Project Overview / Description

SentinelShield is a lightweight educational Web Application Firewall (WAF) and intrusion monitoring framework designed to demonstrate detection and mitigation of common web-based attacks using rule-based filtering and traffic analysis. In the modern threat landscape, web applications represent the primary channel for data exchange, commercial transactions, and user access. Consequently, they are the most targeted surfaces for malicious actors. Traditional firewalls operating at Layers 3 and 4 of the OSI model filter packets based on IP addresses and TCP/UDP ports. However, modern application-layer threats (such as SQL Injection and Cross-Site Scripting) tunnel directly through valid HTTP/HTTPS channels (ports 80/443). As a result, network-level security controls fail to inspect the content of these connections, leaving backend database servers and client browsers vulnerable to compromise. This creates an urgent need for specialized Web Application Firewalls (WAF) that parse, decode, and analyze application-layer data in real time.

SentinelShield provides a realistic, practical platform for students to learn how HTTP request interception, signature-based filtering, log formatting, and rate limiting operate in real-world defensive environments. The system acts as a middleware layer that sits directly in front of a Python Flask web application. It intercepts every incoming HTTP request prior to application routing. It parses every component of the transaction—including request paths, query parameters (GET), form data and JSON payloads (POST), cookies, and metadata headers like User-Agent and Referer. This comprehensive extraction ensures that malicious payloads hidden in obscure fields are not missed by the detection engine.

Additionally, SentinelShield normalizes the extracted data, recursively decoding URL encodings and HTML entities to reveal the raw malicious payload. The parsed request is evaluated against a customizable regular expression ruleset. If an attack signature is identified, the system blocks the request, returns an HTTP 403 Forbidden response, logs the incident, and updates the telemetry dashboard in real time. For traffic control, it tracks request velocity per IP address using a sliding window log queue. Clients exceeding predefined request thresholds are dynamically blocked

with an HTTP 429 Too Many Requests response. This transparent, modular architecture provides students with an in-depth, hands-on look at defensive security engineering.

## 2. Project Scope and Practical Objectives

The primary objective of the SentinelShield practical curriculum is to bridge the gap between theoretical knowledge of web vulnerabilities (such as the OWASP Top 10) and hands-on defensive engineering. Rather than treating security controls as a closed-source black box, students inspect and modify the regular expression rules, study request parsing, and analyze log databases.

By the end of these practical sessions, students are expected to:

- Understand how modern WAFs detect threats using signature patterns, string normalizations, and cascading rule engines.
- Analyze raw HTTP requests, understanding headers, query parameters, cookies, user agents, and POST body encodings.
- Identify and verify common web attacks using manual command-line utilities (like curl) and automated security validation scripts.
- Evaluate rule accuracy by identifying, documenting, and mitigating false positives and false negatives through signature tuning.
- Examine and correlate structured security logs to perform basic incident analysis and reconstruct attack paths.
- Explain the operational workflow of: Request Interception -> Decoding -> Rule Matching -> Logging -> Alerting -> Dashboard Telemetry.
- Configure and validate rate-limiting systems to block brute-force credential stuffing and volumetric resource flooding.

This project is designed to emulate a realistic work environment where students configure tools, analyze data, interpret results, and produce documentation similar to real cybersecurity operations.

### **3. Practical Concepts and Background**

Understanding the mechanics of SentinelShield requires a foundation in application-layer protocols, signature matching, and stateful traffic tracking. This chapter details the technical concepts behind the framework.

#### **3.1 HTTP Request Inspection**

HTTP is a stateless, text-based application protocol. Every client request contains three core components: the Request Line (defining the HTTP method, the URI, and protocol version), the Headers (providing metadata key-value pairs), and the Request Body (containing payload data). Defensive systems must inspect all three layers, as attackers hide payloads in multiple fields to bypass simple filters.

**Request Line:** The first line of an HTTP request. It contains the HTTP method (GET, POST, etc.), the request URI, and the HTTP version. Attackers target the URI itself (e.g., directory traversal via '../' or SQL injection in query parameters).

**Headers:** Key-value pairs containing metadata about the request. Attackers spoof headers like User-Agent (e.g., using scanning bots like Nikto or sqlmap), Referer, or inject SQL payloads into Session Cookies.

**Request Body:** Standard POST requests send data formatted as URL-encoded forms or JSON blocks. The WAF must parse these bodies before they reach the backend database interpreter.

### 3.2 Attack Signature Identification

Signature-based detection matches incoming traffic against a database of known malicious patterns (signatures). In SentinelShield, signatures are written as regular expressions (regex). Writing high-quality signatures requires balancing precision (to avoid blocking legitimate traffic) and coverage (to catch all variations of an exploit). For example:

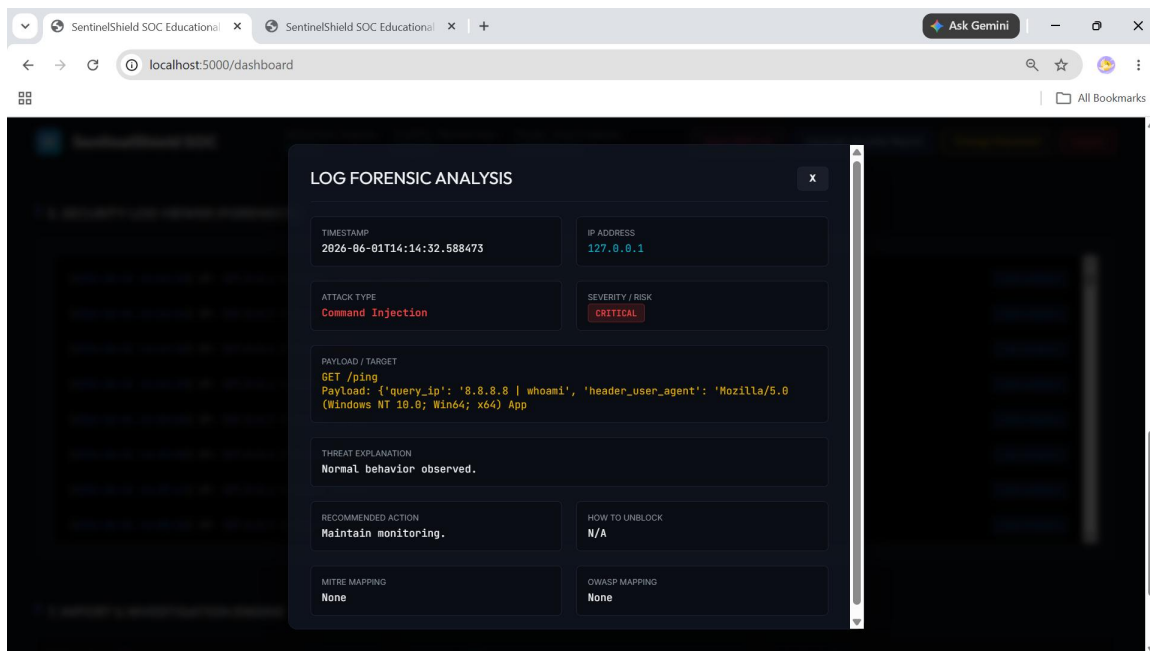
- SQL Injection: Pattern matches SQL keywords ('SELECT', 'UNION', 'INSERT'), database comments ('--', '#'), or tautologies ('1='1').
- Cross-Site Scripting: Pattern matches HTML tags ('<script>'), browser event handlers ('onerror', 'onload'), or protocol handlers ('javascript:').
- Local File Inclusion: Pattern matches directory traversal sequences ('../', '..\') and sensitive filenames ('/etc/passwd', 'win.ini').

### 3.3 Behavior Monitoring and Rate Limiting

Signature matching cannot block brute-force or flooding attacks, as the requests look like standard logins. Instead, the WAF monitors client behavior. The sliding window log algorithm records a history of request timestamps for each client IP. When a new request arrives, the system removes records older than the window size (e.g., 60 seconds) and calculates the current rate. If the rate exceeds the threshold, subsequent requests from that IP are blocked for a lockout duration.

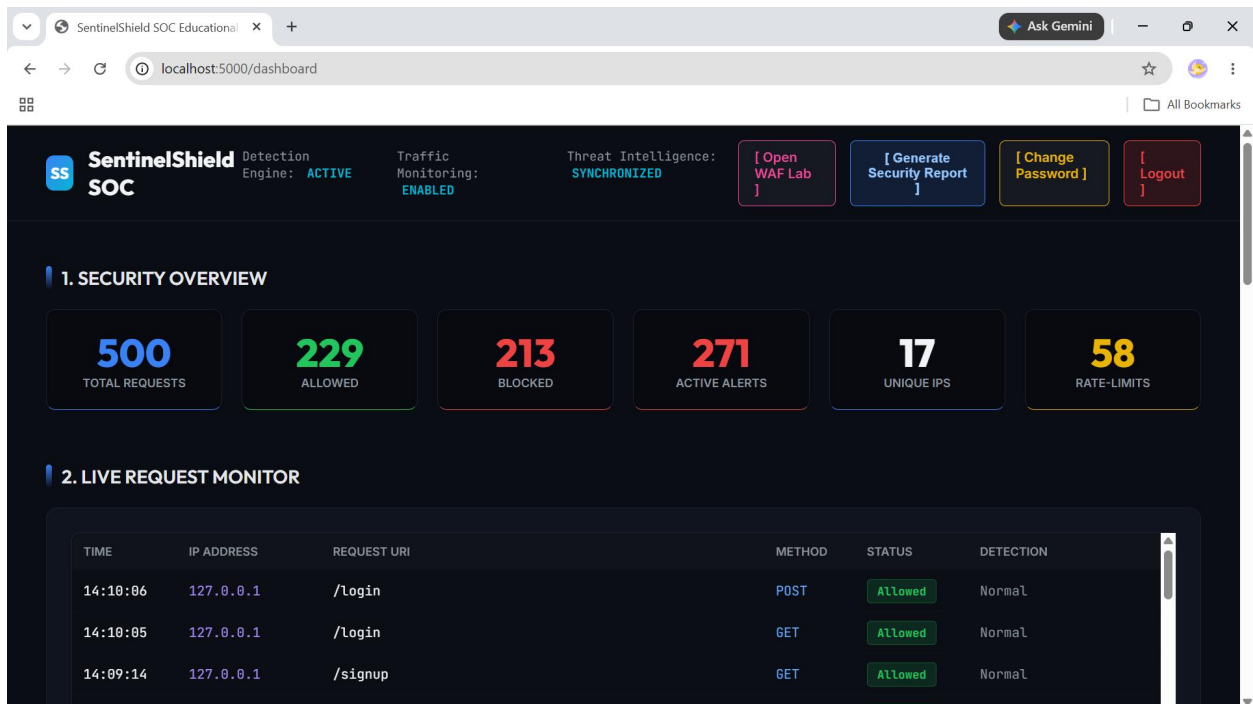
### 3.4 Log Analysis

Log analysis is critical for incident response, threat hunting, and security auditing. SentinelShield generates structured JSON and CSV logs. JSON logs allow automated SIEM parsers to ingest and categorize events, while CSV logs let analysts inspect traffic in spreadsheet tools. Each log includes the timestamp, source IP, target URI, matched rules, threat score, and outcome.



### 3.5 Dashboard Interpretation

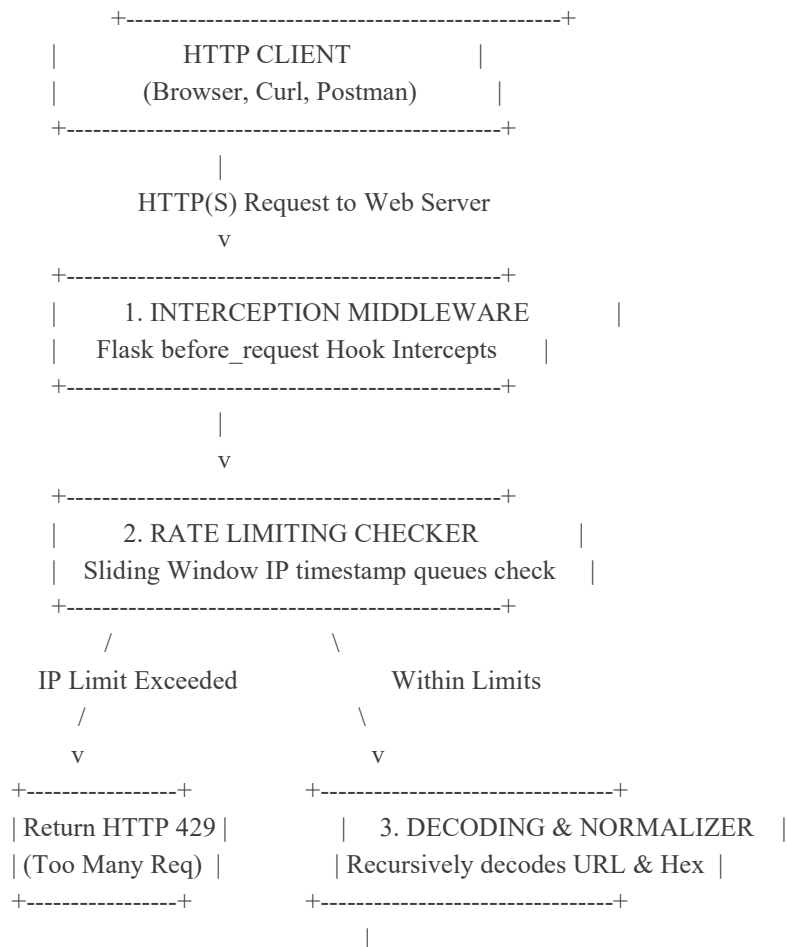
The SentinelShield Dashboard visualizes security data, providing real-time telemetry. Key metrics include total request count, blocked request count, rate-limit violations, and threat distribution charts. The live ledger shows recent events, helping students trace how attacks are blocked and immediately visualized.

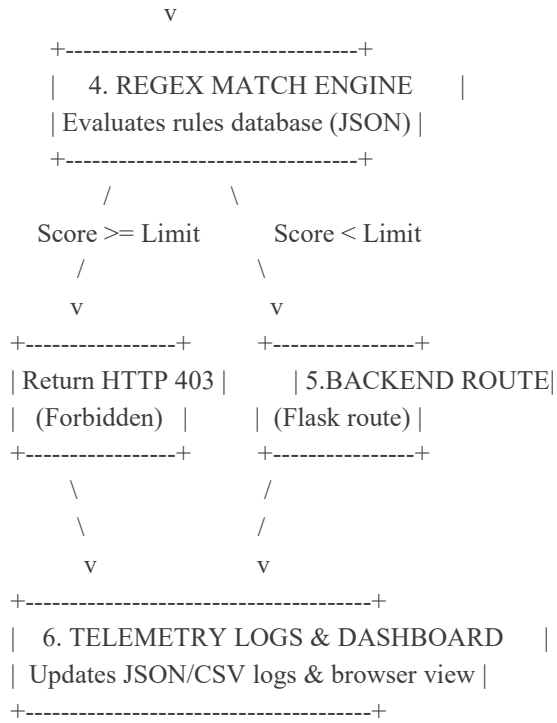


## 4. Practical Workflow

### 4.1 Step 1: Understanding the System Architecture

Students map the flow of HTTP requests through the framework, verifying the sequence of security checks. The data flow path follows an incoming client request as it passes through the rate limiting layer, the normalization layer, the regex rule matching engine, and finally onto the application controller (or the block/warning response generator).





## 4.2 Step 2: Reviewing Rule Definitions

Inspect the custom signature database file (`rules.json`). This file defines the regex patterns, target scopes (headers, query, body), threat scores, and actions. To learn how threat scores accumulate to determine if a request should be blocked.

```
{
  "rule_id": "SEC-SQLI-001",
  "description": "Detects classic SQL Union-based query injections",
  "regex": "(?i)(UNION\\s+SELECT|UNION\\s+ALL\\s+SELECT)",
  "threat_score": 5,
  "action": "BLOCK"
}
```

## 4.3 Step 3: Simulating HTTP Requests

Students use command-line utilities (like `curl`) to send normal and malicious requests to the application. They observe how the WAF handles each request. Test cases must include:

- Benign queries to confirm the application processes standard inputs correctly.
- SQLi payloads to verify signature blocks on parameters and bodies.
- XSS script tags to verify script filter actions.
- Command injections to test shell metacharacter blocking.
- High-frequency script loops to trigger rate-limiting blocks.

## 4.4 Step 4: Observing Detection Behavior

Students document how the application responds to blocked requests, verifying that the WAF returns JSON responses detailing the matched rule, the reason for the block, and the HTTP status code (403 or 429).

## 4.5 Step 5: Log File Examination

Inspect `security_alerts.json` and `access_traffic.csv` to trace security events. They correlate timestamps, source IPs, and triggered rules to understand incident details.

## 5. Detailed System Working

This section explains how SentinelShield operates behind the scenes to inspect and filter web traffic. It describes the pipeline elements in a clear, logical manner.

### 5.1 Request Processing

When a request arrives, the system interprets every part of it: URL, parameters, headers, and optional body. Before running checks, the system normalizes the data, decoding URL encodings (%27 -> ') and HTML entities so obfuscated payloads are revealed. For example, if an attacker sends %3Cscript%3E, the normalizer decodes it to <script> before passing it to the rule engine.

### 5.2 Rule-Based Detection

Each rule reflects a known attack or technique. When the system finds content matching these patterns, it marks the request as suspicious or malicious. To provide academic rigor, each attack category is mapped directly to standard industry vulnerability catalogs (OWASP Top 10:2021 and CWE):

#### 5.2.1 SQL Injection (SQLi)

- OWASP Classification: OWASP Top 10:2021-A03:Injection
- CWE Identifier: CWE-89 (Improper Neutralization of Special Elements used in an SQL Command)
- Regex Signature Pattern: (?(i)(UNION\s+SELECT|' OR\s+.\*=.\*|--|#)
- Description: Attackers inject database commands into input fields to bypass authentication or extract sensitive data.
- Risk Score & Impact: Critical (9/10). Direct threat of database exposure and credential theft.

#### 5.2.2 Cross-Site Scripting (XSS)

- OWASP Classification: OWASP Top 10:2021-A03:Injection
- CWE Identifier: CWE-79 (Improper Neutralization of Input During Web Page Generation)
- Regex Signature Pattern: (?(i)(<script.\*?>|onerror\s\*|=|onload\s\*=)

- **Description:** Attackers inject malicious scripts to execute in target browsers, stealing cookies or session tokens.
- **Risk Score & Impact:** High (7/10). Risk of session hijacking and account takeovers.

### 5.2.3 Local File Inclusion (LFI)

- OWASP Classification: OWASP Top 10:2021-A05:Security Misconfiguration
- CWE Identifier: CWE-22 (Improper Limitation of a Pathname to a Restricted Directory)
- Regex Signature Pattern: `(?i)(\.\.\.\.\./etc/passwd)`
- Description: Attackers manipulate file path parameters to read sensitive system configuration files.
- Risk Score & Impact: High (8/10). Threat of exposing sensitive credentials and system files.

### 5.2.4 Remote File Inclusion (RFI)

- OWASP Classification: OWASP Top 10:2021-A05:Security Misconfiguration
- CWE Identifier: CWE-98 (Improper Control of Generation of Code)
- Regex Signature Pattern: `(?i)(https?|ftp)://[^\s/$.?\#].[\^\s]*` (inside file params)
- Description: Attackers force the web app to execute a remote script hosted on an external server.
- Risk Score & Impact: Critical (9/10). Direct route to remote code execution.

### 5.2.5 Directory Traversal

- OWASP Classification: OWASP Top 10:2021-A01:Broken Access Control
- CWE Identifier: CWE-22 (Path Traversal)
- Regex Signature Pattern: `(?i)(\.\./|\.\.\\)`
- Description: Navigating the directory structure to access unauthorized system folders.
- Risk Score & Impact: Medium-High (6/10). Leads to information disclosure.

### 5.2.6 Command Injection

- OWASP Classification: OWASP Top 10:2021-A03:Injection
- CWE Identifier: CWE-78 (Improper Neutralization of Special Elements used in an OS Command)
- Regex Signature Pattern: `(?i)([;&'\n]|whoami|uname|net\s+user|cat\s+)`
- Description: Injecting shell metacharacters to run operating system commands on the server.

- Risk Score & Impact: Critical (10/10). Complete server compromise.

### **5.2.7 Brute-Force and Credential Stuffing**

- OWASP Classification: OWASP Top 10:2021-A07:Identification and Authentication Failures
- CWE Identifier: CWE-307 (Improper Restriction of Excessive Authentication Attempts)
- Description: Automated scripts systematically submit login credentials to guess password combinations.
- Risk Score & Impact: High (7/10). Leads to account takeovers.

### **5.2.8 Bot Traffic and Port Scanning**

- OWASP Classification: OWASP Top 10:2021-A05:Security Misconfiguration
- CWE Identifier: CWE-1021
- Regex Signature Pattern: (?i)(sqlmap|nikto|dirbuster|gobuster|nmap)
- Description: Scanning tools probe the application for vulnerabilities.
- Risk Score & Impact: Medium (5/10). Reconnaissance phase before targeted attacks.

### **5.2.9 Flooding Attacks**

- OWASP Classification: OWASP Top 10:2021-A05:Security Misconfiguration
- CWE Identifier: CWE-400 (Uncontrolled Resource Consumption)
- Description: Flooding the application with high volumes of requests to exhaust system resources.
- Risk Score & Impact: High (7/10). Denial of service risk.

## **5.3 Traffic Monitoring & Sliding Windows**

The system keeps track of how many requests each IP sends in a certain period. If an IP crosses the allowed threshold, it is considered abusive. SentinelShield manages this using a sliding window algorithm. For each incoming request, the timestamp is recorded in a queue associated with the client's IP. Timestamps older than the sliding window size (60 seconds) are removed. If the queue size exceeds the threshold, subsequent requests from that IP are blocked for the lockout duration.

## **5.4 Alert Decisions Sequence**

Once a request is marked as malicious or abusive, the system follows a decision sequence:

1. The request may be blocked: The connection is terminated, and the WAF returns an HTTP 403 or 429 status code.
2. The event is logged: The incident details are stored in the JSON and CSV databases.
3. The IP may be temporarily or permanently flagged: The IP status is updated in the registry.
4. An alert is generated for dashboard visibility: The telemetry updates in real-time.

## **5.5 Logging and Dashboard Visibility**

- Every blocked or flagged request appears in the log.
- The dashboard summarizes key insights such as attack counts, types, and IP details.
- Students analyze these outputs during practical sessions.

## 6. Practical Output

During laboratory execution, we compile real-world logs and telemetry records. The practical work produces several observable outputs:

- Attempted attack requests and their detection results.
- System messages indicating whether each request was allowed or blocked.
- Log entries containing timestamps, IP addresses, and detection categories.

Summary Observation Table:

| Test Case      | Method | Target URI | Input Payload                 | Matched Rule | Threat Score | Status | Outcome      |
|----------------|--------|------------|-------------------------------|--------------|--------------|--------|--------------|
| Normal Query   | GET    | /index     | None                          | None         | 0            | 200 OK | Allowed      |
| SQLi Bypass    | POST   | /login     | user=admin' OR '1'='1         | SEC-SQLI-001 | 5            | 403    | Blocked      |
| SQLi Union     | GET    | /search    | q=1 UNION SELECT 1,2          | SEC-SQLI-002 | 5            | 403    | Blocked      |
| XSS Script     | GET    | /search    | q=<script>alert(1)</script>   | SEC-XSS-001  | 5            | 403    | Blocked      |
| XSS Error      | GET    | /search    | q=<img src=x onerror=alert()> | SEC-XSS-002  | 4            | 403    | Blocked      |
| Path Traversal | GET    | /view      | file=../../etc/passwd         | SEC-LFI-002  | 4            | 403    | Blocked      |
| Cmd Injection  | GET    | /ping      | ip=127.0.0.1; whoami          | SEC-CMD-001  | 5            | 403    | Blocked      |
| Tool Scan      | GET    | /index     | UA Header: Nikto/2.1.5        | SEC-BOT-001  | 3            | 403    | Blocked      |
| Brute Force    | POST   | /login     | 15 login attempts in 10s      | None         | 0            | 429    | Rate Limited |

## 7. Core Architecture Walkthrough & Method-by-Method Breakdown

To ensure students fully understand the implementation code, this section provides an in-depth code walkthrough of each module. The system uses a middleware-based design to inspect traffic before application routing occurs.

1. **WafMiddleware Class:** Sit in front of Flask's request loop. It binds `self.before_request` to Flask's `app.before_request` hook. This hook is triggered automatically before any route controller executes. The `before_request` method extracts request metadata and checks the source IP against the `RateLimiter` module.
2. **Extraction and Normalization:** If the rate limiter allows the request, `WafMiddleware` extracts parameters from three scopes: `request.args` (query string variables), `request.form` (standard body key-values), and `request.headers` (metadata strings). Each string is passed to `self.normalize()`, which decodes URL-escaped entities and HTML tags to prevent signature bypasses.
3. **Signature Matching:** Sanitized parameters are passed to `self.rule_engine.evaluate()`. The engine compiles and runs the predefined regular expressions. If any parameter triggers a match, it accumulates the threat score. If the threat score reaches or exceeds 4, `WafMiddleware` blocks the request, returning an HTTP 403 Forbidden Response to the client.
4. **RateLimiter Class:** Uses a deque structure for each IP address. When a request is evaluated, `is_allowed()` retrieves the client's timestamp history, prunes entries older than 60 seconds, and counts the active connections. If the count exceeds the threshold (10), it returns `False`, and `WafMiddleware` triggers an HTTP 429 response.
5. **SecurityLogger Class:** Handles audit trails. `log_event()` records transaction details into `security_alerts.json` (using structured JSON) and `access_traffic.csv`. This structured logging makes it easy for security tools to parse and analyze the system's threat data.